PCIe Gen 6 Data Link Layer VIP

Back-to-Back LPIF Verification Architecture

AlexUni x SiVision GP Verification Team

Alexandria University - Faculty of Engineering
Department of Electronics & Communication

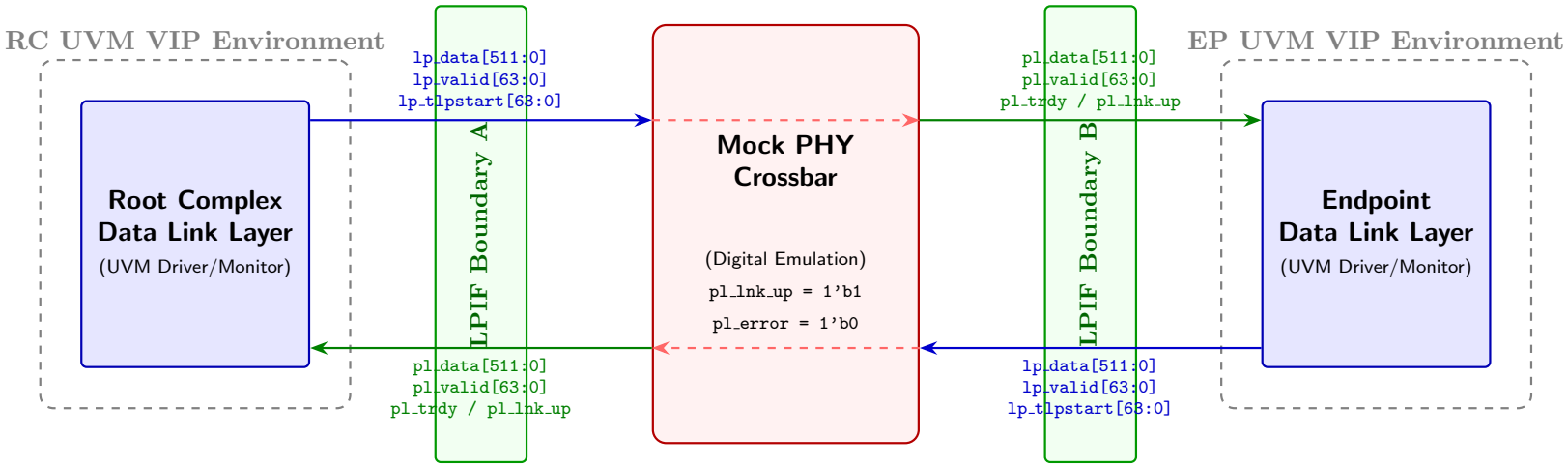
Under the supervision of:
Dr. Mohamed Elbanna

March 2, 2026

Verification Plan

- **Challenge:** Full PCIe stack verification gets bogged down in analog SerDes modeling and complex physical training.
- **Our Approach:** Isolate the Data Link Layer (DLL) to enable rapid, iterative prototyping of protocol logic.
- **The Goal:** Achieve a functional, testable VIP immediately by bypassing physical layer complexity.
- **The Enabler:** Utilizing a standard Logical PHY Interface (LPIF) in a Back-to-Back (B2B) configuration.

Architecture: LPIF VIP-to-VIP Utilization



LPIF Interface

```
interface pcie_lpif_if #(
    parameter NBYTES = 64 // Gen5 x16 (512 bits)
)(
    input logic lclk,
    input logic rst_n
);

    // Tx Path (DLL -> PHY) : "lp_" signals
    logic [(NBYTES*8)-1:0] lp_data;
    logic [NBYTES-1:0] lp_valid;
    logic [7:0] lp_stream;
    logic lp_irdy;
    logic [3:0] lp_state_req;
    //...
```

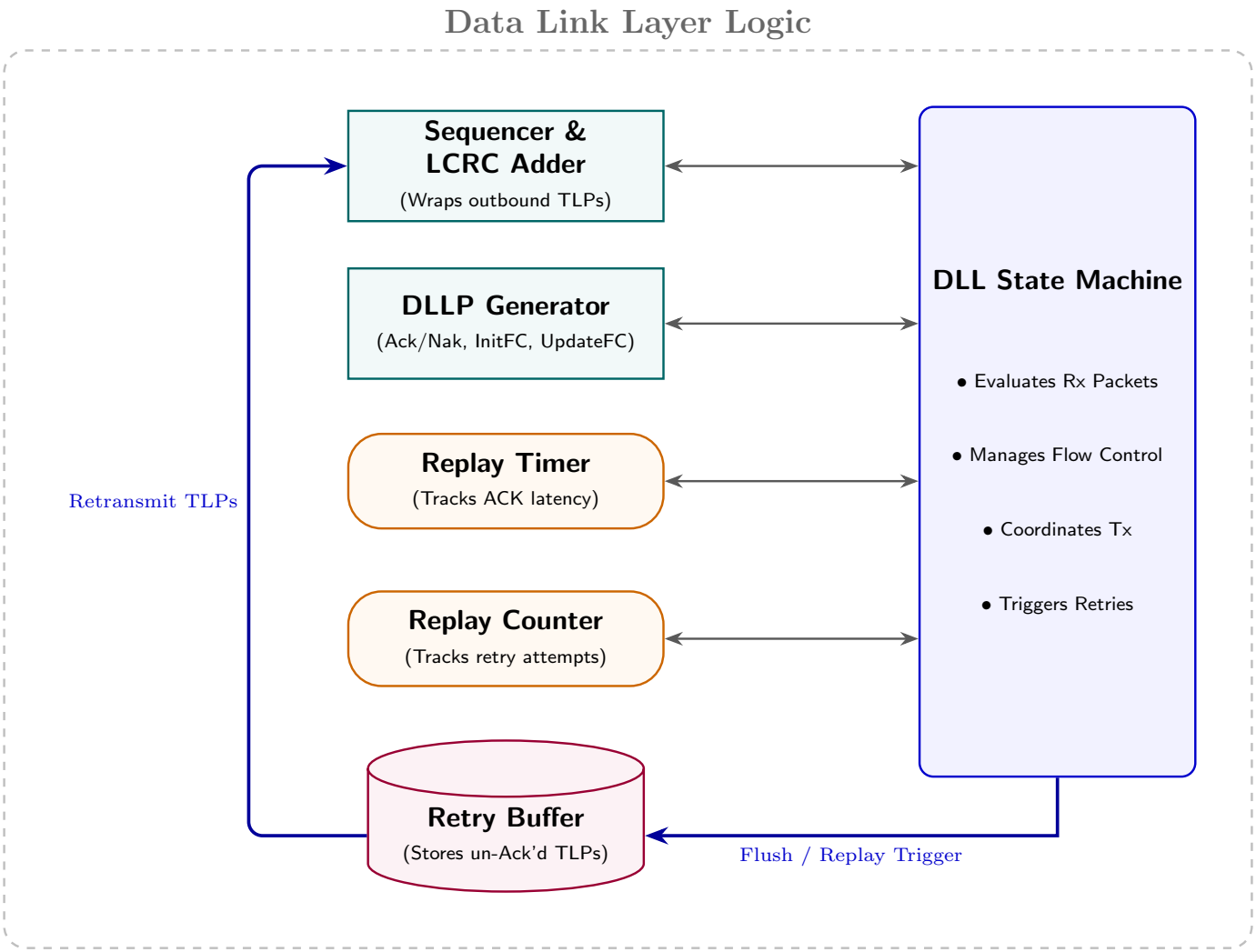
LPIF Interface

```
//...
// Rx Path (PHY -> DLL) : "pl_" signals
logic [(NBYTES*8)-1:0] pl_data;
logic [NBYTES-1:0]     pl_valid;
logic [7:0]            pl_stream;
logic                  pl_trdy;
logic [3:0]            pl_state_sts;
logic                  pl_lnk_up;
//..
```

1. The LPIF VIP-to-VIP Utilization

- **The Boundary:** The VIP encapsulates strictly the DLL, using LPIF as the digital boundary.
- **Lightweight Emulation:** The LPIF acts as a lightweight logical PHY, allowing us to route data without analog overhead.
- **The Mock-Phy Crossbar:** A custom RTL module connects the Root Complex (RC) and Endpoint (EP) VIPs.
- **Strategic Tie-Offs:**
  - We force `pl_lnk_up = 1` and `pl_error = 0` to bypass LTSSM training.
  - This tricks the state machines into an active state on clock cycle 1, enabling immediate packet testing.

Architecture: Internal Data Link Layer Logic

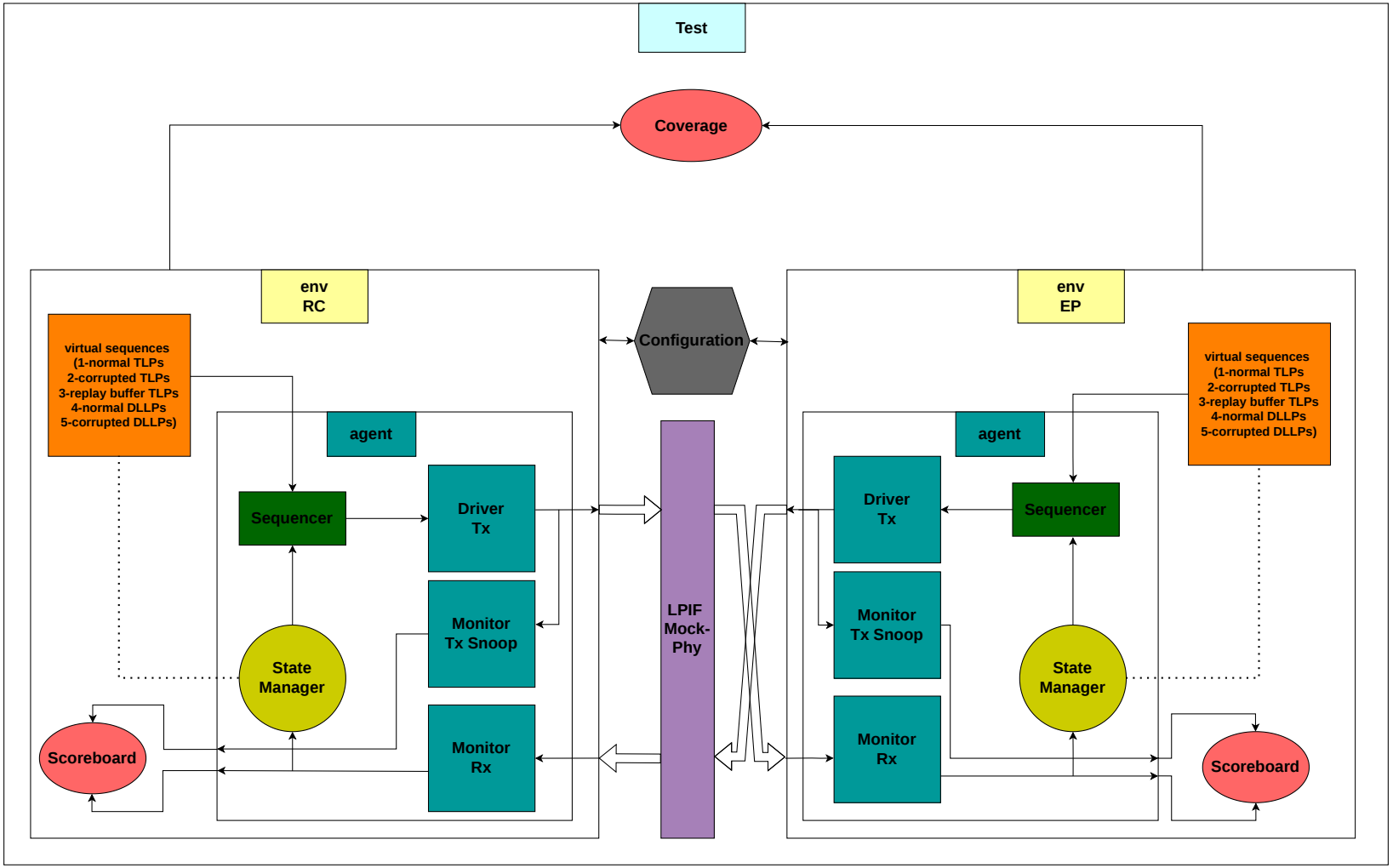


2. Internal Data Link Layer Logic

Core UVM Components within the VIP:

- **State Machine:** The central brain tracking DL_Init, DL_Active, and Flow Control initialization.
- **Sequencer & LCRC Adder:** Wraps outbound Transaction Layer Packets (TLPs) with 12-bit sequence numbers and 32-bit LCRC.
- **DLLP Generator:** Reactively constructs ACKs, NAKs, and Flow Control updates based on Rx monitor triggers.
- **Retry Buffer:** A queue storing un-Ack'd TLPs for potential retransmission.
- **Replay Timers & Counters:** Enforce strict PCIe protocol limits on ACK latency and retry attempts.

Architecture: Proposed UVM Architecture (Draft)



3. Proposed UVM Architecture

A Symmetrical, Loopback Testbench:

- **Top-Level Isolation:** The RTL `mock_phy` sits in `top.sv`, keeping the UVM software completely hardware-agnostic.
- **Dual Environments:** The base test instantiates two identical `pcie_env` blocks, configured independently as RC and EP.
- **Dual Monitors:**
  - *Tx Snoop:* Captures what the Driver puts on the bus.
  - *Rx Monitor:* Captures what arrives from the LPIF Mock-Phy.
- **Centralized Scoreboard:** Compares Tx Snoop and Rx Monitor data to verify sequence numbers, LCRC packing, and successful ACK/NAK replays.

Next Steps: Detailed Verification Plan

Blueprint for UVM Implementation:

- **1. Scenarios & Coverage:** Map critical edge cases (e.g., credit exhaustion, NAK storms) and define strict functional coverage metrics.
- **2. Sequences:** Design the hierarchy of virtual sequences covering link initialization (InitFC), standard traffic, and error injection.
- **3. Assertions (SVA):** Define continuous protocol checks at the LPIF boundary (e.g., blocking TLP transmission if credit counters are zero).